

Transaction Management In Spring Boot

thegeekplanets

Table of Contents

1. What is a transaction and what does ACID stand for?	3
2. What is the @Transactional annotation and where can you place it?	3
3. How do you enable transaction management in Spring Boot?	4
4. What is the default transaction propagation behavior in Spring?.....	4
5. What happens when an exception occurs inside a transactional method?	5
6. What is the difference between checked and unchecked exceptions in transaction rollback?	6
7. What happens if you call a transactional method from another method in the same class?	7
8. How do you configure a method to rollback on specific exceptions?.....	8
9. What is a TransactionManager and why is it needed?	9
10. What is the difference between programmatic and declarative transaction management?	10
11. How do you rollback a transaction manually?	11
12. What is the readOnly attribute in @Transactional and when should you use it?	12
13. Explain the key transaction propagation types: REQUIRED, REQUIRES_NEW, and NESTED.	13
14. What are transaction isolation levels and what problems do they solve (dirty read, non- repeatable read, phantom read)?	15
15. What is the difference between PROPAGATION_REQUIRED and PROPAGATION_REQUIRES_NEW with a practical example?	17
16. How does the timeout attribute work in @Transactional?	21
17. How do you configure multiple transaction managers in Spring Boot?.....	23
18. What is optimistic locking vs pessimistic locking in the context of transactions?	27
19. Why doesn't @Transactional work with self-invocation and how do you solve it?.....	31
20: What happens when you have nested @Transactional methods with different propagation levels?	31
21. What are the performance implications of different isolation levels?.....	33

22. How do you handle transactions when using both JPA and another data source (like MongoDB)?	33
23. What is connection pooling and how does it relate to transaction management?	34
24. How do you handle transactions with asynchronous methods (@Async)?	35
25. What is transaction synchronization and when would you use it?	36
26. How do you design transaction boundaries in a microservices architecture?	37
27. What is the Saga pattern and when would you use it over traditional transactions?	38
28. How do you handle distributed transactions across multiple databases?	40
29. How do you implement compensating transactions?	41
30. What strategies can you use to avoid transaction timeouts?	42
31. How do you implement retry logic for failed transactions?	44
32. How do you ensure data integrity across multiple services without distributed transactions?	45
33. What is the Unit of Work pattern and how does Spring implement it?	47
34. What are the anti-patterns in transaction management and how do you prevent them?	48
35. What transaction-related issues do you commonly identify in code reviews and how do you mentor developers to avoid them?	50
36. How do you make architectural decisions about when to use strong consistency (transactions) vs eventual consistency in a new project?	52
Thank You! 🙏	55

1. What is a transaction and what does ACID stand for?

Transaction: A transaction is a sequence of database operations that are treated as a single unit of work. Either all operations succeed (commit) or all fail (rollback), ensuring data consistency.

ACID Properties:

- **Atomicity:** All operations in a transaction complete successfully or none do. It's "all or nothing."
 - **Consistency:** The database moves from one valid state to another, maintaining all defined rules and constraints.
 - **Isolation:** Concurrent transactions don't interfere with each other. Each transaction executes as if it's the only one running.
 - **Durability:** Once a transaction is committed, the changes are permanent, even in case of system failure.
-

2. What is the @Transactional annotation and where can you place it?

What it is: @Transactional is a declarative annotation that tells Spring to manage transactions automatically for the annotated method or class.

Where to place it:

- **Method level:** Applies to that specific method only
- **Class level:** Applies to all public methods in the class
- **Interface level:** Applies to all implementing classes (though not recommended)

```
// Class level
@Transactional
public class UserService {
    // All public methods are transactional
}

// Method level
public class UserService {
    @Transactional
    public void createUser(User user) {
        // Only this method is transactional
    }
}
```

Best Practice: Place it at the service layer, not in controllers or repositories.

3. How do you enable transaction management in Spring Boot?

In Spring Boot: Transaction management is **auto-configured** by default when you include spring-boot-starter-data-jpa or similar starters. No additional configuration needed.

What happens automatically:

- Spring Boot creates a PlatformTransactionManager bean
- Enables @Transactional annotation processing
- Configures proxy-based transaction management

Manual configuration (if needed):

```
@SpringBootApplication
@EnableTransactionManagement // Usually not needed in Spring Boot
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

For custom TransactionManager:

```
@Bean
public PlatformTransactionManager transactionManager(EntityManagerFactory emf) {
    return new JpaTransactionManager(emf);
}
```

4. What is the default transaction propagation behavior in Spring?

Default: PROPAGATION_REQUIRED

What it means: If a transaction already exists, the method joins it. If no transaction exists, a new one is created.

```
@Transactional // Default is PROPAGATION_REQUIRED
public void methodA() {
    // Transaction T1 starts here
    methodB(); // Joins T1
}

@Transactional // Also PROPAGATION_REQUIRED
public void methodB() {
    // Runs in same transaction as methodA
}
```

Behavior:

- Both methods share the same transaction
 - If methodB throws an exception, the entire transaction (including methodA's work) rolls back
 - Most common and suitable for majority of use cases
-

5. What happens when an exception occurs inside a transactional method?

Depends on the exception type:**Unchecked Exceptions (RuntimeException and Error):**

- Transaction **automatically rolls back**
- All database changes are undone

Checked Exceptions:

- Transaction **commits by default** (does NOT rollback)
- Database changes are saved

```
@Transactional
public void processOrder(Order order) {
    orderRepository.save(order);

    // This causes automatic rollback
    throw new RuntimeException("Payment failed");
}

@Transactional
public void processOrder(Order order) throws IOException {
    orderRepository.save(order);

    // This DOES NOT rollback by default
    throw new IOException("File not found");
}
```

Key Point: The exception must propagate out of the transactional method boundary for rollback to occur. If you catch and handle it internally, no rollback happens.

6. What is the difference between checked and unchecked exceptions in transaction rollback?

Aspect	Unchecked Exceptions	Checked Exceptions
Types	RuntimeException, Error	Exception (except RuntimeException)
Default Rollback	YES	NO
Examples	NullPointerException, IllegalArgumentException	IOException, SQLException
Compilation	Not required to catch/declare	Must catch or declare

Examples:

```

@Transactional
public void example1() {
    userRepository.save(user);
    throw new NullPointerException(); // Rolls back
}

@Transactional
public void example2() throws IOException {
    userRepository.save(user);
    throw new IOException(); // Does NOT rollback
}

// To force rollback on checked exceptions:
@Transactional(rollbackFor = IOException.class)
public void example3() throws IOException {
    userRepository.save(user);
    throw new IOException(); // NOW it rolls back
}

// To prevent rollback on unchecked exceptions:
@Transactional(noRollbackFor = IllegalArgumentException.class)
public void example4() {
    userRepository.save(user);
    throw new IllegalArgumentException(); // Does NOT rollback
}

```

7. What happens if you call a transactional method from another method in the same class?

The `@Transactional` annotation doesn't work! This is called the self-invocation problem.

Why it happens: Spring uses **proxies** to implement transaction management. When you call a method from within the same class, you're bypassing the proxy and calling the actual object directly.

```
@Service
public class UserService {

    public void publicMethod() {
        // This is a direct call, NOT through proxy
        this.transactionalMethod(); // @Transactional is IGNORED!
    }

    @Transactional
    private void transactionalMethod() {
        // Transaction won't start here
        userRepository.save(user);
    }
}
```

Solutions:

1. Inject self (Spring 4.3+):

```
@Service
public class UserService {
    @Autowired
    private UserService self;

    public void publicMethod() {
        self.transactionalMethod(); // Works!
    }

    @Transactional
    public void transactionalMethod() {
        userRepository.save(user);
    }
}
```

2. Move to separate service:

```
@Service
public class UserTransactionService {
    @Transactional
    public void transactionalMethod() {
        userRepository.save(user);
    }
}
```



```
@Service
public class UserService {
    @Autowired
    private UserTransactionService txService;

    public void publicMethod() {
        txService.transactionalMethod(); // Works!
    }
}
```

8. How do you configure a method to rollback on specific exceptions?

Use the `rollbackFor` and `noRollbackFor` attributes:

Rollback on specific checked exceptions:

```
@Transactional(rollbackFor = {IOException.class, SQLException.class})
public void processFile() throws IOException {
    // Will rollback if IOException or SQLException occurs
}
```

Rollback on any Exception:

```
@Transactional(rollbackFor = Exception.class)
public void processData() throws Exception {
    // Rolls back on both checked and unchecked exceptions
}
```

Prevent rollback on specific exceptions:

```
@Transactional(noRollbackFor = ValidationException.class)
public void validateAndSave(User user) {
    // Won't rollback if ValidationException occurs
    // Will still rollback on other RuntimeExceptions
}
```

Combine both:

```
@Transactional(
    rollbackFor = Exception.class,
    noRollbackFor = {ValidationException.class, BusinessException.class}
)
public void complexOperation() {
    // Rolls back on all exceptions EXCEPT the specified ones
}
```

9. What is a TransactionManager and why is it needed?

What it is: TransactionManager is Spring's abstraction for managing transactions across different transaction APIs (JPA, JDBC, Hibernate, JTA, etc.).

Main interface: PlatformTransactionManager

Why it's needed:

- Provides a **consistent programming model** regardless of the underlying technology
- Abstracts away the complexity of different transaction APIs
- Manages transaction lifecycle: begin, commit, rollback
- Handles transaction context and thread safety

Common implementations:

```
// For JPA/Hibernate
@Bean
public PlatformTransactionManager transactionManager(EntityManagerFactory emf) {
    return new JpaTransactionManager(emf);
}

// For JDBC
@Bean
public PlatformTransactionManager transactionManager(DataSource dataSource) {
    return new DataSourceTransactionManager(dataSource);
}

// For distributed transactions (JTA)
@Bean
public PlatformTransactionManager transactionManager() {
    return new JtaTransactionManager();
}
```

In Spring Boot: Automatically configured based on your dependencies.

10. What is the difference between programmatic and declarative transaction management?

Declarative Transaction Management

Using annotations - Spring manages transactions automatically

```
@Transactional
public void transferMoney(Account from, Account to, BigDecimal amount) {
    from.debit(amount);
    to.credit(amount);
    accountRepository.save(from);
    accountRepository.save(to);
    // Spring automatically commits or rolls back
}
```

Advantages:

- Clean, readable code
- No transaction boilerplate
- Easier to maintain
- Recommended approach

Disadvantages:

- Less control over transaction boundaries
- Limited programmatic control

Programmatic Transaction Management

Manually managing transactions using TransactionTemplate or PlatformTransactionManager

```
@Autowired
private TransactionTemplate transactionTemplate;

public void transferMoney(Account from, Account to, BigDecimal amount) {
    transactionTemplate.execute(status -> {
        try {
            from.debit(amount);
            to.credit(amount);
            accountRepository.save(from);
            accountRepository.save(to);
            return true;
        } catch (Exception e) {
            status.setRollbackOnly();
            return false;
        }
    });
}
```

Advantages:

- Fine-grained control over transaction boundaries
- Can make dynamic decisions within transaction
- Useful for complex scenarios

Disadvantages:

- More boilerplate code
- Harder to maintain
- Mixes business logic with transaction code

When to use programmatic:

- Need dynamic transaction control
- Complex nested transaction logic
- Partial rollbacks or savepoints
- When declarative approach is too restrictive

11. How do you rollback a transaction manually?

Method 1: Throw an unchecked exception

```
@Transactional
public void processOrder(Order order) {
    orderRepository.save(order);

    if (!paymentService.validate(order)) {
        throw new RuntimeException("Payment validation failed");
    }
}
```

Method 2: Using TransactionAspectSupport

```
@Transactional
public void processOrder(Order order) {
    orderRepository.save(order);

    if (!paymentService.validate(order)) {
        TransactionAspectSupport.currentTransactionStatus().setRollbackOnly();
        return;
    }
}
```

Method 3: Programmatic with TransactionTemplate

```
@Autowired
private TransactionTemplate transactionTemplate;

public void processOrder(Order order) {
    transactionTemplate.execute(status -> {
        orderRepository.save(order);

        if (!paymentService.validate(order)) {
            status.setRollbackOnly();
        }
        return null;
    });
}
```

Important Notes:

- Once setRollbackOnly() is called, the transaction MUST rollback, even if no exception occurs
- You cannot undo setRollbackOnly() - it's a one-way flag
- The transaction will still appear to complete normally but changes won't be committed

12. What is the readOnly attribute in @Transactional and when should you use it?

What it is: A hint to the persistence provider that the transaction will only perform read operations, no writes.

```
@Transactional(readOnly = true)
public List<User> getAllUsers() {
    return userRepository.findAll();
}

@Transactional(readOnly = true)
public User getUserById(Long id) {
    return userRepository.findById(id)
        .orElseThrow(() -> new UserNotFoundException(id));
}
```

Benefits:**Performance Optimization:**

- Hibernate can skip dirty checking for entities
- Some databases optimize read-only transactions
- May skip flushing the session
- JDBC driver optimizations possible

Resource Management:

- Database can use read-only connections from connection pool
- Some databases allow read replicas for read-only transactions

When to use:

- Query methods that only fetch data
- Report generation
- Search operations
- Any method that doesn't modify database state

When NOT to use:

```
@Transactional(readOnly = true) // WRONG!
public void updateUser(User user) {
    userRepository.save(user); // This may fail or be ignored
}
```

Important: If you try to perform write operations in a read-only transaction, behavior depends on the database and JPA provider. Some will throw an exception, others might silently ignore writes.

13. Explain the key transaction propagation types: REQUIRED, REQUIRES_NEW, and NESTED.

PROPAGATION_REQUIRED (Default)

Behavior: Join existing transaction or create new one if none exists.

```
@Transactional(propagation = Propagation.REQUIRED)
public void methodA() {
    // Transaction T1 starts
    methodB(); // Joins T1
}

@Transactional(propagation = Propagation.REQUIRED)
public void methodB() {
    // Runs in same transaction T1
}
```

Key Points:

- Both methods share the same transaction
- If methodB fails, entire transaction rolls back
- Most commonly used propagation type
- Good for operations that must succeed or fail together

PROPAGATION_REQUIRES_NEW

Behavior: Always creates a new transaction, suspending the current one if it exists.

```

@Transactional(propagation = Propagation.REQUIRED)
public void methodA() {
    // Transaction T1 starts
    saveAuditLog(); // Creates T2, suspends T1
    // T1 resumes
}

@Transactional(propagation = Propagation.REQUIRES_NEW)
public void saveAuditLog() {
    // New Transaction T2 starts
    auditRepository.save(log);
    // T2 commits independently
}

```

Key Points:

- Creates physically separate transactions
- Inner transaction commits/rolls back independently
- If outer transaction fails, inner transaction stays committed
- Useful for audit logging that must persist regardless of main operation
- Can lead to database deadlocks if not careful

Use Cases:

- Audit logging that must succeed even if main operation fails
- Sending notifications
- Recording metrics or statistics

PROPAGATION_NESTED

Behavior: Creates a nested transaction (savepoint) within the existing transaction.

```

@Transactional(propagation = Propagation.REQUIRED)
public void methodA() {
    // Transaction T1 starts
    try {
        methodB(); // Creates savepoint within T1
    } catch (Exception e) {
        // Can continue, only methodB's changes rolled back
    }
    // T1 can still commit
}

```

```
@Transactional(propagation = Propagation.NESTED)
public void methodB() {
    // Nested transaction (savepoint) within T1
}
```

Key Points:

- Uses JDBC savepoints (not all databases support this)
- Nested transaction can rollback independently
- If outer transaction rolls back, nested transaction also rolls back
- If nested transaction fails, outer transaction can still commit
- Only works with DataSourceTransactionManager

Differences from REQUIRES_NEW:

- NESTED uses savepoints (single physical transaction)
- REQUIRES_NEW creates separate physical transaction
- NESTED is rolled back if parent rolls back
- REQUIRES_NEW commits independently

Database Support: PostgreSQL, Oracle, SQL Server support savepoints. MySQL InnoDB supports them too.

14. What are transaction isolation levels and what problems do they solve (dirty read, non-repeatable read, phantom read)?

Transaction Isolation Levels control how changes made by one transaction are visible to other concurrent transactions.

The Three Concurrency Problems:

1. Dirty Read: Transaction reads data written by another uncommitted transaction.

```
// Transaction 1
@Transactional
public void updateBalance() {
    account.setBalance(1000); // Not yet committed
}

// Transaction 2 (at same time)
@Transactional
public void readBalance() {
    int balance = account.getBalance(); // Reads 1000 (dirty read!)
    // But Transaction 1 might rollback!
}
```

Problem: If Transaction 1 rolls back, Transaction 2 read invalid data.

2. Non-Repeatable Read: Transaction reads the same row twice and gets different values because another transaction modified and committed it.

```
@Transactional
public void processAccount() {
    int balance1 = account.getBalance(); // Reads 1000

    // Another transaction updates balance to 500 and commits

    int balance2 = account.getBalance(); // Reads 500
    // Same query, different result!
}
```

Problem: Data inconsistency within a single transaction.

3. Phantom Read: Transaction executes same query twice and gets different number of rows because another transaction inserted or deleted rows.

```
@Transactional
public void calculateTotal() {
    List<Order> orders1 = orderRepository.findByStatus("PENDING");
    // Count: 5 orders

    // Another transaction inserts new PENDING order and commits

    List<Order> orders2 = orderRepository.findByStatus("PENDING");
    // Count: 6 orders (phantom row appeared!)
}
```

Problem: Result set changes within same transaction.

Isolation Levels in Spring:

READ_UNCOMMITTED (Lowest isolation):

@Transactional(isolation = Isolation.READ_UNCOMMITTED)

- Allows: Dirty reads, non-repeatable reads, phantom reads
- Fastest but least safe
- Rarely used in production

READ_COMMITTED:

@Transactional(isolation = Isolation.READ_COMMITTED)

- Prevents: Dirty reads
- Allows: Non-repeatable reads, phantom reads
- Default for many databases (PostgreSQL, Oracle, SQL Server)

REPEATABLE_READ:

@Transactional(isolation = Isolation.REPEATABLE_READ)

- Prevents: Dirty reads, non-repeatable reads
- Allows: Phantom reads
- Default for MySQL InnoDB

SERIALIZABLE (Highest isolation):

@Transactional(isolation = Isolation.SERIALIZABLE)

- Prevents: All three problems
- Slowest performance (transactions execute serially)
- Use only when absolute consistency required

DEFAULT:

@Transactional(isolation = Isolation.DEFAULT)

- Uses database's default isolation level

15. What is the difference between PROPAGATION_REQUIRED and PROPAGATION_REQUIRES_NEW with a practical example?

PROPAGATION_REQUIRED Example:

```
@Service
public class OrderService {

    @Transactional(propagation = Propagation.REQUIRED)
    public void placeOrder(Order order) {
        // Transaction T1 starts
        orderRepository.save(order);

        updateInventory(order); // Joins T1

        // If updateInventory fails, ENTIRE order rolls back
    }
}
```

```

@Transactional(propagation = Propagation.REQUIRED)
public void updateInventory(Order order) {
    // Uses same Transaction T1
    for (OrderItem item : order.getItems()) {
        inventoryRepository.decreaseStock(item.getProductId(), item.getQuantity());
    }
}
}

```

Outcome with REQUIRED:

- Both methods share Transaction T1
- If inventory update fails, order save also rolls back
- All-or-nothing behavior
- Single database transaction

PROPAGATION_REQUIRES_NEW Example:

```

@Service
public class OrderService {

    @Autowired
    private NotificationService notificationService;

    @Transactional(propagation = Propagation.REQUIRED)
    public void placeOrder(Order order) {
        // Transaction T1 starts
        orderRepository.save(order);

        try {
            notificationService.sendOrderConfirmation(order); // Creates T2
        } catch (Exception e) {
            // Log error but don't fail order
            log.error("Notification failed", e);
        }

        // T1 commits even if notification failed
    }
}

@Service
public class NotificationService {

    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void sendOrderConfirmation(Order order) {
        // New Transaction T2 starts (T1 suspended)
        NotificationLog log = new NotificationLog();
        log.setOrderId(order.getId());
    }
}

```

```

        log.setStatus("SENT");
        notificationLogRepository.save(log);

        emailService.send(order.getCustomer().getEmail(), "Order Confirmed");
        // T2 commits independently
        // T1 resumes
    }
}

```

Outcome with REQUIRES_NEW:

- Order uses Transaction T1
- Notification uses separate Transaction T2
- If T1 fails, T2 stays committed (notification log saved)
- If T2 fails, T1 can still commit (order still processed)
- Two separate database transactions

Practical Comparison:

Use REQUIRED when:

- Operations must succeed or fail together
- Strong consistency required
- Parent-child relationship (order and order items)
- Financial transactions

Use REQUIRES_NEW when:

- Independent operations that shouldn't affect each other
- Audit logging (must persist regardless of main operation)
- Notification/event logging
- Background tasks
- Cross-cutting concerns

Real-world scenario:

```

@Service
public class PaymentService {

    @Transactional
    public void processPayment(Payment payment) {
        // Main transaction T1
        payment.setStatus("PROCESSING");
        paymentRepository.save(payment);

        // Audit should succeed even if payment fails
        auditService.logPaymentAttempt(payment); // Uses REQUIRES_NEW

        // Process payment
        boolean success = externalPaymentGateway.charge(payment);

        if (!success) {
            throw new PaymentFailedException();
            // Payment rolls back, but audit log remains
        }

        payment.setStatus("COMPLETED");
        paymentRepository.save(payment);
    }
}

```

```

@Service
public class AuditService {

    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void logPaymentAttempt(Payment payment) {
        // Independent transaction T2
        AuditLog log = new AuditLog();
        log.setPaymentId(payment.getId());
        log.setTimestamp(LocalDateTime.now());
        auditLogRepository.save(log);
        // T2 commits here, regardless of payment outcome
    }
}

```

16. How does the timeout attribute work in @Transactional?

What it is: Specifies maximum time (in seconds) a transaction can run before automatically rolling back.

```
@Transactional(timeout = 30) // 30 seconds
public void processLargeReport() {
    // If this takes longer than 30 seconds, transaction rolls back
    reportRepository.generateReport();
}
```

How it works:

- Timer starts when transaction begins
- If timeout expires, Spring throws TransactionTimedOutException
- Transaction automatically rolls back
- Default timeout is underlying transaction system's default (usually no timeout)

Practical Examples:

Preventing long-running transactions:

```
@Transactional(timeout = 10)
public void quickOperation() {
    // Must complete within 10 seconds
    userRepository.save(user);
}

@Transactional(timeout = 300) // 5 minutes
public void batchProcessing() {
    // Longer timeout for batch operations
    for (Record record : records) {
        processRecord(record);
    }
}
```

With propagation:

```
@Transactional(timeout = 60)
public void parentMethod() {
    // 60 second timeout for entire transaction
    childMethod(); // Shares same timeout
}

@Transactional(propagation = Propagation.REQUIRES_NEW, timeout = 10)
public void childMethod() {
    // Creates new transaction with its own 10-second timeout
}
```

Important Considerations:

1. Timeout applies to entire transaction:

```
@Transactional(timeout = 5)
public void complexOperation() {
    step1(); // Takes 2 seconds
    step2(); // Takes 2 seconds
    step3(); // Takes 2 seconds
    // Total 6 seconds - will timeout and rollback!
}
```

2. Doesn't interrupt running queries:

- Timeout is checked between operations, not during
- If a single query takes longer than timeout, it might complete
- Database-level query timeout is separate

3. Resolution varies by transaction manager:

- Some check only at transaction boundaries
- Others check more frequently
- JDBC timeout is more granular than JPA timeout

Best Practices:

```
// Set reasonable timeouts based on operation type
@Service
public class UserService {

    @Transactional(timeout = 5) // Short for simple CRUD
    public User createUser(User user) {
        return userRepository.save(user);
    }

    @Transactional(timeout = 60) // Longer for complex operations
    public void generateMonthlyReport() {
        List<Transaction> transactions = transactionRepository.findByMonth(month);
        reportGenerator.process(transactions);
    }

    @Transactional(timeout = 300) // Even longer for batch
    public void importUsers(List<User> users) {
        userRepository.saveAll(users);
    }
}
```

Use timeouts to:

- Prevent database connection starvation
- Detect and fail fast on stuck operations
- Protect against infinite loops or performance issues
- Enforce SLAs (Service Level Agreements)

17. How do you configure multiple transaction managers in Spring Boot?

Scenario: When you have multiple data sources (e.g., one MySQL for users, one PostgreSQL for orders), you need separate transaction managers.

Step 1: Configure Multiple Data Sources

```
@Configuration
public class DataSourceConfig {

    // Primary DataSource (MySQL)
    @Primary
    @Bean(name = "mysqlDataSource")
    @ConfigurationProperties(prefix = "spring.datasource.mysql")
    public DataSource mysqlDataSource() {
        return DataSourceBuilder.create().build();
    }

    // Secondary DataSource (PostgreSQL)
    @Bean(name = "postgresDataSource")
    @ConfigurationProperties(prefix = "spring.datasource.postgres")
    public DataSource postgresDataSource() {
        return DataSourceBuilder.create().build();
    }
}
```

application.yml:

```
spring:
  datasource:
    mysql:
      url: jdbc:mysql://localhost:3306/userdb
      username: root
      password: secret
      driver-class-name: com.mysql.cj.jdbc.Driver
    postgres:
      url: jdbc:postgresql://localhost:5432/orderdb
      username: postgres
      password: secret
      driver-class-name: org.postgresql.Driver
```


Step 2: Configure Entity Managers

```
@Configuration
@EnableJpaRepositories(
    basePackages = "com.example.repository.mysql",
    entityManagerFactoryRef = "mysqlEntityManagerFactory",
    transactionManagerRef = "mysqlTransactionManager"
)
public class MySQLConfig {

    @Primary
    @Bean(name = "mysqlEntityManagerFactory")
    public LocalContainerEntityManagerFactoryBean mysqlEntityManagerFactory(
        @Qualifier("mysqlDataSource") DataSource dataSource,
        EntityManagerFactoryBuilder builder) {
        return builder
            .dataSource(dataSource)
            .packages("com.example.entity.mysql")
            .persistenceUnit("mysql")
            .build();
    }

    @Primary
    @Bean(name = "mysqlTransactionManager")
    public PlatformTransactionManager mysqlTransactionManager(
        @Qualifier("mysqlEntityManagerFactory") EntityManagerFactory emf) {
        return new JpaTransactionManager(emf);
    }
}
```

```
@Configuration
@EnableJpaRepositories(
    basePackages = "com.example.repository.postgres",
    entityManagerFactoryRef = "postgresEntityManagerFactory",
    transactionManagerRef = "postgresTransactionManager"
)
public class PostgresConfig {

    @Bean(name = "postgresEntityManagerFactory")
    public LocalContainerEntityManagerFactoryBean postgresEntityManagerFactory(
        @Qualifier("postgresDataSource") DataSource dataSource,
        EntityManagerFactoryBuilder builder) {
        return builder
            .dataSource(dataSource)
            .packages("com.example.entity.postgres")
            .persistenceUnit("postgres")
            .build();
    }
}
```

```

@Bean(name = "postgresTransactionManager")
public PlatformTransactionManager postgresTransactionManager(
    @Qualifier("postgresEntityManagerFactory") EntityManagerFactory emf) {
    return new JpaTransactionManager(emf);
}
}

```

Step 3: Use Specific Transaction Managers

```

@Service
public class UserService {

    @Autowired
    private UserRepository userRepository; // MySQL repository

    // Uses primary (MySQL) transaction manager by default
    @Transactional
    public User createUser(User user) {
        return userRepository.save(user);
    }

    // Or explicitly specify
    @Transactional(transactionManager = "mysqlTransactionManager")
    public User updateUser(User user) {
        return userRepository.save(user);
    }
}

```

```

@Service
public class OrderService {

    @Autowired
    private OrderRepository orderRepository; // PostgreSQL repository

    // Must specify non-primary transaction manager
    @Transactional(transactionManager = "postgresTransactionManager")
    public Order createOrder(Order order) {
        return orderRepository.save(order);
    }
}

```

Step 4: Using Both Transaction Managers

```

@Service
public class BusinessService {

    @Autowired
    private UserService userService;

    @Autowired
    private OrderService orderService;

    // Two separate transactions - no atomicity between them!
    public void createUserAndOrder(User user, Order order) {
        userService.createUser(user);        // MySQL transaction
        orderService.createOrder(order);      // PostgreSQL transaction
        // If order creation fails, user is still created!
    }

    // For distributed transaction, need JTA
    @Transactional(transactionManager = "jtaTransactionManager")
    public void createUserAndOrderAtomically(User user, Order order) {
        userService.createUser(user);
        orderService.createOrder(order);
        // Both succeed or both fail (requires JTA setup)
    }
}

```

Important Notes:

1. One must be @Primary:

- Required when multiple transaction managers exist
- Used when @Transactional doesn't specify which manager

2. Repository packages must be separate:

- Each transaction manager manages specific repositories
- Repositories in different packages avoid conflicts

3. No automatic distributed transactions:

- Each transaction manager manages its own transaction independently
- For atomic operations across databases, need JTA (Java Transaction API)

4. ChainedTransactionManager (deprecated):

- Previously used for pseudo-distributed transactions
- No longer recommended - use proper distributed transaction solution instead

18. What is optimistic locking vs pessimistic locking in the context of transactions?

Both are concurrency control mechanisms to handle simultaneous data access, but they work differently.

Optimistic Locking

Philosophy: Assumes conflicts are rare, so don't lock the data. Check for conflicts before committing.

How it works:

- Read data along with a version number
- When updating, check if version is still the same
- If version changed, someone else modified it - throw exception
- If version same, update and increment version

Implementation in JPA:

```
@Entity
public class Product {
    @Id
    private Long id;

    private String name;
    private BigDecimal price;

    @Version // Optimistic locking version field
    private Long version;

    // getters and setters
}
```

What happens:

```
@Service
public class ProductService {

    @Transactional
    public void updatePrice(Long productId, BigDecimal newPrice) {
        // 1. Reads product with version = 1
        Product product = productRepository.findById(productId).get();

        // 2. User modifies price
        product.setPrice(newPrice);

        // 3. Save generates SQL:
        // UPDATE product SET price = ?, version = 2
        // WHERE id = ? AND version = 1
    }
}
```

```

        // If version is no longer 1 (someone else updated it),
        // throws OptimisticLockException
        productRepository.save(product);
    }
}

```

Handling conflicts:

```

@Service
public class ProductService {

    public void updatePriceWithRetry(Long productId, BigDecimal newPrice) {
        int maxRetries = 3;
        int attempt = 0;

        while (attempt < maxRetries) {
            try {
                updatePrice(productId, newPrice);
                return; // Success
            } catch (OptimisticLockException e) {
                attempt++;
                if (attempt >= maxRetries) {
                    throw new RuntimeException("Failed after " + maxRetries + " attempts",
e);
                }
                // Retry with fresh data
            }
        }
    }

    @Transactional
    private void updatePrice(Long productId, BigDecimal newPrice) {
        Product product = productRepository.findById(productId).get();
        product.setPrice(newPrice);
        productRepository.save(product);
    }
}

```

Pessimistic Locking

Philosophy: Assumes conflicts are common, so lock the data immediately to prevent conflicts.

How it works:

- Acquire database lock when reading data
- Other transactions must wait until lock is released
- No conflicts because only one transaction can access at a time

Types of Pessimistic Locks:**PESSIMISTIC_READ (Shared Lock):**

```
@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {

    @Lock(LockModeType.PESSIMISTIC_READ)
    @Query("SELECT p FROM Product p WHERE p.id = :id")
    Optional<Product> findByIdWithReadLock(@Param("id") Long id);
}
```

Multiple transactions can read simultaneously, But no one can write until all read locks released.

PESSIMISTIC_WRITE (Exclusive Lock):

```
@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {

    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("SELECT p FROM Product p WHERE p.id = :id")
    Optional<Product> findByIdWithWriteLock(@Param("id") Long id);
}
```

Only one transaction can access (read or write), Others must wait.

Usage example:

```
@Service
public class ProductService {

    @Transactional
    public void updatePriceWithLock(Long productId, BigDecimal newPrice) {
        // Acquires exclusive lock - others wait
        Product product = productRepository.findByIdWithWriteLock(productId).get();

        product.setPrice(newPrice);
        // Lock automatically released when transaction commits
        productRepository.save(product);
    }
}
```

Comparison and When to Use

Optimistic Locking:

Pros:

- Better performance (no database locks)
- No deadlock risk
- Scalable for high concurrency
- Works well with long-running transactions

Cons:

- Transaction can fail at commit time
- Requires retry logic
- User might lose work if conflict occurs

Best for:

- Low conflict scenarios
- Read-heavy applications
- Web applications with long user think time
- When performance is critical

Pessimistic Locking:

Pros:

- Guarantees no conflicts
- Predictable behavior
- No need for retry logic

Cons:

- Worse performance (waiting for locks)
- Risk of deadlocks
- Reduces concurrency
- Can cause timeouts

Best for:

- High conflict scenarios
- Critical sections (like inventory management)
- Financial transactions
- When data consistency is more important than performance

19. Why doesn't @Transactional work with self-invocation and how do you solve it?

The Problem

@Transactional doesn't work when calling methods within the same class because Spring uses **proxy-based AOP** (Aspect-Oriented Programming) to implement transaction management.

How Spring transactions work:

1. Spring creates a proxy wrapper around your service class
2. The proxy intercepts method calls and adds transaction logic
3. When you call a method from outside, it goes through the proxy
4. When you call a method from inside the same class, it bypasses the proxy

Visual representation:

External call: Client → Proxy (adds transaction) → Target Object → Method

Internal call: Client → Proxy → Target Object → this.method() (NO PROXY!)

Key takeaway: The problem occurs because Spring's default proxy mechanism can't intercept internal method calls. The solution is to ensure transactional methods are called through the Spring proxy.

20: What happens when you have nested @Transactional methods with different propagation levels?

The behavior depends on the propagation type combination:

REQUIRED (parent) + REQUIRED (nested):

- Both share the same transaction
- If either fails, everything rolls back

REQUIRED (parent) + REQUIRES_NEW (nested):

- Nested method creates a new independent transaction
- Parent transaction is suspended
- Nested commits/rolls back independently of parent
- Parent can continue even if nested fails (if exception is caught)

REQUIRED (parent) + NESTED (nested):

- Nested method creates a savepoint within parent transaction
- Nested can rollback to savepoint without affecting parent
- Parent rollback also rolls back nested changes

REQUIRED (parent) + MANDATORY (nested):

- Nested joins parent transaction (same as REQUIRED)

REQUIRED (parent) + NOT_SUPPORTED (nested):

- Nested runs without transaction
- Parent transaction is suspended during nested execution

REQUIRED (parent) + NEVER (nested):

- Throws exception because nested explicitly forbids running in a transaction

Common use cases:

```
// Audit log must persist even if main operation fails
@Transactional(propagation = Propagation.REQUIRED)
public void mainOperation() {
    doWork();
    auditService.log(); // Uses REQUIRES_NEW - always saves
}

// Optional operation that shouldn't fail the main transaction
@Transactional(propagation = Propagation.REQUIRED)
public void mainOperation() {
    doWork();
    try {
        optionalService.process(); // Uses NESTED - can fail independently
    } catch (Exception e) {
        // Continue with main operation
    }
}
```

Key principle: The nested method's propagation type determines whether it joins the existing transaction, creates a new one, or modifies the transaction behavior.

21. What are the performance implications of different isolation levels?

Higher isolation = More locking = Worse performance

Performance ranking (fastest to slowest):

- **READ_UNCOMMITTED:** Fastest, no read locks, allows dirty reads
- **READ_COMMITTED:** Good balance, most common default
- **REPEATABLE_READ:** Slower, holds locks longer
- **SERIALIZABLE:** Slowest, maximum locking, transactions run serially

Typical performance impact:

- **READ_COMMITTED:** Baseline (100%)
- **REPEATABLE_READ:** 30-50% slower
- **SERIALIZABLE:** 70-90% slower

Best practice: Use READ_COMMITTED by default, increase only for critical operations.

22. How do you handle transactions when using both JPA and another data source (like MongoDB)?

Each data source needs its own transaction manager. They operate independently - no automatic ACID across both.

```
// Separate transaction managers
@Transactional(transactionManager = "jpaTransactionManager")
public void saveToMySQL(User user) { }

@Transactional(transactionManager = "mongoTransactionManager")
public void saveToMongo(Activity activity) { }

// Two independent transactions - if one fails, other still commits!
public void saveBoth(User user, Activity activity) {
    saveToMySQL(user);    // Commits independently
    saveToMongo(activity); // Commits independently
}
```

Solutions:

- **Transactional Outbox Pattern:** Save event in MySQL, background job syncs to MongoDB
- **Saga Pattern:** Manual compensation if second transaction fails
- **Eventual Consistency:** Accept temporary inconsistency
- **JTA (avoid if possible):** Complex distributed transaction coordinator

23. What is connection pooling and how does it relate to transaction management?

Connection pooling reuses database connections instead of creating new ones for each transaction.

Key relationship:

- Each transaction requires one connection from the pool
- Transaction holds connection until commit/rollback
- Pool size limits concurrent transactions

```
spring.datasource.hikari:
  maximum-pool-size: 10 # Max 10 concurrent transactions
  connection-timeout: 30000 # Wait time if pool exhausted
```

Common issues:

- Pool exhaustion: All connections in use, new transactions wait or timeout
- Connection leak: Long transactions hold connections, starving other requests

```
@Transactional(timeout = 30) // Release connection after 30s
public void avoidHoldingConnection() {
    //
}
```

Best practice: Pool size = (cores × 2) + disk_count (typically 10-20 for web apps)

24. How do you handle transactions with asynchronous methods (@Async)?

Key rule: Transactions don't propagate to @Async methods because they run in different threads.

```
@Transactional
public void parentMethod() {
    orderRepository.save(order);
    asyncService.processAsync(order); // Runs in separate thread!
    // Transaction commits here, before async completes
}

@Async
@Transactional // Separate transaction in new thread
public void processAsync(Order order) {
    // New transaction, independent of parent
}

Problem scenario:
@Transactional
public void createOrder() {
    Order order = orderRepository.save(order); // ID = null until commit
    asyncService.updateOrder(order.getId());    // May execute before commit!
}

@Async
public void updateOrder(Long id) {
    Order order = orderRepository.findById(id); // Might not exist yet!
}
```

Solutions:

1. Use TransactionSynchronization (recommended):

```
@Transactional
public void createOrder() {
    Order order = orderRepository.save(order);

    TransactionSynchronizationManager.registerSynchronization(
        new TransactionSynchronization() {
            @Override
            public void afterCommit() {
                asyncService.processOrder(order.getId()); // After commit!
            }
        }
    );
}
```

2. Use @TransactionalEventListener:

```
@Transactional
public void createOrder() {
    Order order = orderRepository.save(order);
    eventPublisher.publishEvent(new OrderCreatedEvent(order.getId()));
}

@Async
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
public void handleOrderCreated(OrderCreatedEvent event) {
    processOrder(event.getOrderid()); // Executes after commit
}
```

3. Separate async method transaction:

```
@Async
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void processOrder(Long orderId) {
    // Independent transaction in async thread
}
```

25. What is transaction synchronization and when would you use it?

Transaction synchronization allows you to register callbacks that execute at specific points in the transaction lifecycle.

Main callbacks:

- beforeCommit() - Before transaction commits
- afterCommit() - After successful commit
- afterCompletion(status) - After commit or rollback

Usage:

```
@Transactional
public void createUser(User user) {
    userRepository.save(user);

    TransactionSynchronizationManager.registerSynchronization(
        new TransactionSynchronization() {
            @Override
            public void afterCommit() {
                emailService.sendWelcomeEmail(user); // After DB commit
                cacheService.clearCache();
            }
        }
    );
}
```

Spring's @TransactionalEventListener is built on this:

```
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
public void handleEvent(UserCreatedEvent event) {
    // Cleaner alternative to manual synchronization
}
```

26. How do you design transaction boundaries in a microservices architecture?

Key principle: Each microservice manages its own database and transactions. No distributed transactions.

Strategies:

1. Saga Pattern (Choreography):

```
// Order Service
@Transactional
public void createOrder(Order order) {
    orderRepository.save(order);
    eventPublisher.publish(new OrderCreatedEvent(order));
}

// Inventory Service (separate microservice)
@TransactionalEventListener
@Transactional
public void handleOrderCreated(OrderCreatedEvent event) {
    try {
        inventoryRepository.decreaseStock(event.getProductId());
        eventPublisher.publish(new InventoryReservedEvent(event.getOrderId()));
    } catch (Exception e) {
        eventPublisher.publish(new InventoryReservationFailedEvent(event.getOrderId()));
    }
}

// Order Service compensates if inventory fails
@TransactionalEventListener
@Transactional
public void handleInventoryFailed(InventoryReservationFailedEvent event) {
    orderRepository.cancelOrder(event.getOrderId());
}
```

2. Saga Pattern (Orchestration):

```
// Saga Orchestrator
public void executeOrderSaga(Order order) {
    try {
        orderService.createOrder(order);           // Step 1
        inventoryService.reserveInventory(order);   // Step 2
        paymentService.processPayment(order);       // Step 3
    } catch (Exception e) {
        // Compensate in reverse order
        paymentService.refund(order);
        inventoryService.releaseInventory(order);
        orderService.cancelOrder(order);
    }
}
```

Transaction boundaries:

- Service boundary = Transaction boundary
- Use eventual consistency between services
- Compensating transactions for failures
- Idempotent operations (can retry safely)

27. What is the Saga pattern and when would you use it over traditional transactions?

Answer:

Saga is a pattern for managing distributed transactions by breaking them into local transactions with compensating actions.

Traditional transaction (single database):

```
@Transactional
public void createOrder() {
    orderRepository.save(order);           // All or nothing
    inventoryRepository.decrease(product);
    paymentRepository.save(payment);
}
```

Saga (distributed services):

```
// Step 1: Order Service
@Transactional
public void createOrder(Order order) {
    orderRepository.save(order);
    publishEvent(new OrderCreated(order));
}

// Step 2: Inventory Service
@Transactional
public void reserveInventory(OrderCreated event) {
    try {
        inventoryRepository.reserve(event.getProduct());
        publishEvent(new InventoryReserved(event));
    } catch (Exception e) {
        publishEvent(new SagaFailed(event)); // Triggers compensation
    }
}

// Compensation: Order Service
@Transactional
public void compensateOrder(SagaFailed event) {
    orderRepository.cancelOrder(event.getOrderId());
}
```

Two types:**1. Choreography (Event-driven):**

- Services listen and react to events
- Decentralized
- Good for simple flows

2. Orchestration (Central coordinator):

```
public class OrderSagaOrchestrator {
    public void execute(Order order) {
        try {
            createOrder(order);
            reserveInventory(order);
            processPayment(order);
        } catch (Exception e) {
            compensate(order); // Rollback in reverse
        }
    }
}
```


When to use Saga over traditional transactions:

- Microservices with separate databases
- Long-running business processes
- Cross-system operations
- When 2PC/XA transactions are too slow or unavailable

When to use traditional transactions:

- Single database
- Monolithic applications
- Operations requiring immediate consistency
- Short-lived operations

28. How do you handle distributed transactions across multiple databases?**Option 1: Avoid them (recommended)**

- Redesign to use single database per transaction
- Use eventual consistency
- Accept temporary inconsistency

Option 2: Two-Phase Commit (2PC) with JTA

```
@Configuration
public class JtaConfig {
    @Bean
    public JtaTransactionManager transactionManager() {
        return new JtaTransactionManager();
    }
}

@Transactional // Uses JTA manager
public void distributedTransaction() {
    db1Repository.save(data1); // Both commit together
    db2Repository.save(data2); // or both rollback
}
```

Downsides of 2PC:

- High latency (two round trips)
- Blocking (locks held during voting)
- Coordinator single point of failure
- Not supported by many modern databases (MongoDB, Cassandra)

Option 3: Saga Pattern (recommended)

```
// Sequential local transactions with compensation
public void processOrder(Order order) {
    db1Transaction(() -> saveOrder(order));
    try {
        db2Transaction(() -> reserveInventory(order));
    } catch (Exception e) {
        db1Transaction(() -> cancelOrder(order)); // Compensate
        throw e;
    }
}
```

Best practice: Use Saga or Outbox pattern. Avoid 2PC unless absolutely necessary.

29. How do you implement compensating transactions?

Compensating transactions undo the effects of completed transactions when a later step fails.

Basic pattern:

```
public class OrderSagaService {

    public void processOrder(Order order) {
        try {
            // Forward steps
            Long orderId = createOrder(order); // Step 1
            reserveInventory(order);           // Step 2
            chargePayment(order);              // Step 3
        } catch (Exception e) {
            // Compensation (reverse order)
            refundPayment(order);              // Compensate Step 3
            releaseInventory(order);           // Compensate Step 2
            cancelOrder(order);                // Compensate Step 1
        }
    }

    @Transactional
    public Long createOrder(Order order) {
        return orderRepository.save(order).getId();
    }

    @Transactional
    public void cancelOrder(Order order) {
        order.setStatus("CANCELLED");
        orderRepository.save(order);
    }
}
```

Key principles:

- Each forward action has a compensating action
- Compensations execute in reverse order
- Must be idempotent (can be retried safely)
- Track state to know what needs compensation

State tracking:

```
@Entity
public class SagaState {
    private Long orderId;
    private List<String> completedSteps; // ["ORDER_CREATED", "INVENTORY_RESERVED"]

    public void addStep(String step) {
        completedSteps.add(step);
    }

    public void compensate() {
        for (String step : Lists.reverse(completedSteps)) {
            executeCompensation(step);
        }
    }
}
```

30. What strategies can you use to avoid transaction timeouts?**Answer:****1. Increase timeout (quick fix):**

```
@Transactional(timeout = 300) // 5 minutes
public void longOperation() {
    //
}
```

2. Break into smaller transactions:

```
// Bad: Single large transaction
@Transactional
public void processBatch(List<Record> records) {
    records.forEach(r -> process(r)); // Holds connection entire time
}

// Good: Multiple small transactions
public void processBatch(List<Record> records) {
    records.forEach(r -> processOne(r)); // Each has own transaction
}
```

```
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void processOne(Record record) {
    recordRepository.save(record);
}
```

3. Use batch processing:

```
@Transactional
public void processBatch(List<Record> records) {
    for (int i = 0; i < records.size(); i += 100) {
        List<Record> batch = records.subList(i, Math.min(i + 100, records.size()));
        recordRepository.saveAll(batch);
        entityManager.flush();
        entityManager.clear(); // Free memory
    }
}
```

4. Move heavy operations outside transaction:

```
@Transactional
public void processOrder(Order order) {
    // Heavy operation outside transaction
    ValidationResult result = expensiveValidation(order);

    // Quick transaction
    if (result.isValid()) {
        orderRepository.save(order);
    }
}
```

5. Use asynchronous processing:

```
@Transactional
public void createOrder(Order order) {
    orderRepository.save(order); // Quick save
    asyncService.processHeavyTask(order.getId()); // Outside transaction
}
```

Other strategies:

- Optimize queries (add indexes, avoid N+1)
- Use read-only transactions for queries
- Connection pool tuning
- Database query timeout separate from transaction timeout

31. How do you implement retry logic for failed transactions?

1. Using Spring Retry (recommended):

```
@Configuration
@EnableRetry
public class RetryConfig { }

@Service
public class OrderService {

    @Retryable(
        value = {OptimisticLockException.class, DeadlockLoserDataAccessException.class},
        maxAttempts = 3,
        backoff = @Backoff(delay = 1000, multiplier = 2)
    )
    @Transactional
    public void updateOrder(Order order) {
        orderRepository.save(order);
    }

    @Recover
    public void recover(Exception e, Order order) {
        log.error("Failed after 3 attempts: {}", order.getId());
        // Fallback logic
    }
}
```

2. Manual retry with exponential backoff:

```
public void updateOrderWithRetry(Order order) {
    int maxAttempts = 3;
    int attempt = 0;
    long delay = 1000;

    while (attempt < maxAttempts) {
        try {
            updateOrder(order);
            return; // Success
        } catch (OptimisticLockException e) {
            attempt++;
            if (attempt >= maxAttempts) throw e;
            Thread.sleep(delay);
            delay *= 2; // Exponential backoff
        }
    }
}

@Transactional
private void updateOrder(Order order) {
    orderRepository.save(order);
}
```

3. Retry only on specific exceptions:

```
@Retryable(
    include = {OptimisticLockException.class, TransientDataAccessException.class},
    exclude = {DataIntegrityViolationException.class}, // Don't retry validation errors
    maxAttempts = 3
)
@Transactional
public void processPayment(Payment payment) { }
```

Key points:

- Only retry transient failures (deadlocks, optimistic locks, network issues)
- Don't retry validation or business logic errors
- Use exponential backoff to avoid thundering herd
- Set maximum attempts to prevent infinite loops

32. How do you ensure data integrity across multiple services without distributed transactions?

1. Saga Pattern with compensations:

```
// Each service has local transaction + compensation
@Transactional
public void createOrder(Order order) {
    orderRepository.save(order);
    eventPublisher.publish(new OrderCreated(order));
}

// If later step fails, compensate
@Transactional
public void cancelOrder(Long orderId) {
    Order order = orderRepository.findById(orderId);
    order.setStatus("CANCELLED");
    orderRepository.save(order);
}
```

2. Idempotency keys:

```
@Entity
public class Order {
    @Column(unique = true)
    private String idempotencyKey; // Prevent duplicate processing
}

@Transactional
public Order createOrder(OrderRequest request) {
    Order existing = orderRepository.findByIdempotencyKey(request.getKey());
    if (existing != null) return existing; // Already processed
}
```

```

    Order order = new Order(request);
    order.setIdempotencyKey(request.getKey());
    return orderRepository.save(order);
}

```

3. Outbox pattern for reliable messaging:

```

@Transactional
public void createOrder(Order order) {
    orderRepository.save(order);
    // Save event in same transaction - guarantees delivery
    outboxRepository.save(new OutboxEvent("OrderCreated", order));
}

```

4. Event sourcing:

```

// Store events instead of state
@Transactional
public void processOrder(OrderCommand command) {
    List<Event> events = order.process(command);
    eventStore.saveAll(events); // All events or none
}

```

5. Optimistic concurrency control:

```

@Entity
public class Account {
    @Version
    private Long version; // Detects concurrent modifications
    private BigDecimal balance;
}

```

Key principles:

- Each service owns its data exclusively
- Use events for cross-service communication
- Make operations idempotent
- Design for eventual consistency
- Track business state explicitly

33. What is the Unit of Work pattern and how does Spring implement it?

Unit of Work tracks changes to objects during a business transaction and coordinates writing changes to the database.

Concept:

- Register all changes in memory
- Execute all writes in single database transaction
- Automatic change detection
- Batch operations for efficiency

Spring/JPA implementation:

1. EntityManager as Unit of Work:

```
@Transactional
public void updateUser(List<User> users) {
    // EntityManager tracks all changes
    for (User user : users) {
        User managed = entityManager.find(User.class, user.getId());
        managed.setName(user.getName()); // Change tracked
        // No explicit save needed
    }
    // On transaction commit, all changes written at once
}
```

2. Automatic dirty checking:

```
@Transactional
public void updateUser(Long id, String newName) {
    User user = userRepository.findById(id).get();
    user.setName(newName); // Change detected automatically
    // No save() call needed - UPDATE executed on commit
}
```

3. First-level cache (identity map):

```
@Transactional
public void example() {
    User user1 = entityManager.find(User.class, 1L); // DB query
    User user2 = entityManager.find(User.class, 1L); // From cache
    assert user1 == user2; // Same object instance
}
```

Benefits:

- Prevents lost updates
- Batches database operations
- Reduces database round trips
- Maintains object identity within transaction

Manual control:

```
entityManager.flush(); // Write changes to DB immediately
entityManager.clear(); // Detach all entities
entityManager.detach(user); // Detach specific entity
```

34. What are the anti-patterns in transaction management and how do you prevent them?**Answer:****1. Transaction-per-Request (too broad):**

```
// Bad: Transaction around entire HTTP request
@Transactional
@GetMapping("/users")
public List<UserDTO> getUsers() {
    List<User> users = userService.findAll();
    return users.stream().map(this::toDTO).collect(toList());
    // Holds connection during DTO conversion!
}

// Good: Transaction only for data access
@GetMapping("/users")
public List<UserDTO> getUsers() {
    List<User> users = userService.findAll(); // Transactional inside
    return users.stream().map(this::toDTO).collect(toList());
}
```

2. Missing transaction boundaries:

```
// Bad: No transaction
public void updateUser(User user) {
    User existing = userRepository.findById(user.getId());
    existing.setEmail(user.getEmail());
    // Changes lost! No transaction to flush
}

// Good: Explicit transaction
@Transactional
public void updateUser(User user) {
    User existing = userRepository.findById(user.getId());
    existing.setEmail(user.getEmail());
}
```

3. Catching exceptions incorrectly:

```
// Bad: Swallows exception, transaction commits
@Transactional
public void processOrder(Order order) {
    try {
        orderRepository.save(order);
        throw new RuntimeException("Error");
    } catch (Exception e) {
        log.error("Error", e); // Transaction still commits!
    }
}

// Good: Rethrow or setRollbackOnly
@Transactional
public void processOrder(Order order) {
    try {
        orderRepository.save(order);
    } catch (Exception e) {
        TransactionAspectSupport.currentTransactionStatus().setRollbackOnly();
        throw e;
    }
}
```

4. Long-running transactions:

```
// Bad: Holds connection for entire operation
@Transactional
public void importUsers(List<User> users) {
    for (User user : users) {
        processUser(user); // Slow processing
    }
}

// Good: Smaller transactions
public void importUsers(List<User> users) {
    for (User user : users) {
        processUserInTransaction(user);
    }
}

@Transactional(propagation = REQUIRES_NEW)
void processUserInTransaction(User user) { }
```

5. Self-invocation (proxy bypass):

```
// Bad: @Transactional ignored
public void methodA() {
    this.methodB(); // Direct call, no proxy
}

@Transactional
public void methodB() { }

// Good: Call through proxy
@Autowired private MyService self;

public void methodA() {
    self.methodB(); // Goes through proxy
}
```

Other anti-patterns:

- Read-only flag on write operations
- Wrong isolation level (SERIALIZABLE for all)
- Not handling optimistic lock exceptions
- Ignoring connection pool exhaustion

35. What transaction-related issues do you commonly identify in code reviews and how do you mentor developers to avoid them?

Common issues:**1. Missing @Transactional:**

```
// Wrong
public void updateUser(User user) {
    User existing = userRepository.findById(user.getId()).get();
    existing.setEmail(user.getEmail());
    // Changes not saved!
}

// Correct
@Transactional
public void updateUser(User user) {
    User existing = userRepository.findById(user.getId()).get();
    existing.setEmail(user.getEmail());
}
```

2. @Transactional on wrong layer:

```
// Wrong: Controller level
@RestController
public class UserController {
    @Transactional // Don't do this
    @GetMapping("/users")
    public List<User> getUsers() { }
}

// Correct: Service level
@Service
public class UserService {
    @Transactional
    public List<User> getUsers() { }
}
```

3. Not specifying rollbackFor on checked exceptions:

```
// Wrong: Won't rollback on checked exception
@Transactional
public void processFile() throws IOException {
    userRepository.save(user);
    throw new IOException(); // Transaction commits!
}

// Correct
@Transactional(rollbackFor = Exception.class)
public void processFile() throws IOException {
    userRepository.save(user);
    throw new IOException(); // Transaction rolls back
}
```

4. LazyInitializationException:

```
// Wrong
@Transactional
public User getUser(Long id) {
    return userRepository.findById(id).get();
}

public void display() {
    User user = service.getUser(1L);
    user.getOrders().size(); // LazyInitializationException!
}

// Correct: Fetch eagerly or use DTO
@Transactional
public User getUserWithOrders(Long id) {
    User user = userRepository.findById(id).get();
    Hibernate.initialize(user.getOrders());
    return user;
}
```

Mentoring approach:

- Code review checklist for transaction boundaries
- Pair programming on complex transaction scenarios
- Share real production incidents
- Brown bag sessions on transaction patterns
- Document team standards

36. How do you make architectural decisions about when to use strong consistency (transactions) vs eventual consistency in a new project?

Decision framework:**Use Strong Consistency (ACID Transactions) when:****1. Financial operations:**

```
// Money transfer - MUST be atomic
@Transactional(isolation = Isolation.SERIALIZABLE)
public void transferFunds(Account from, Account to, BigDecimal amount) {
    from.debit(amount);
    to.credit(amount);
    // Both or neither
}
```

2. Inventory with oversell risk:

```
// Stock reservation - MUST prevent overselling
@Transactional
public void reserveProduct(Long productId, int quantity) {
    Product product = productRepository.findByIdWithLock(productId);
    if (product.getStock() < quantity) throw new OutOfStockException();
    product.decreaseStock(quantity);
}
```

3. Critical regulatory/compliance data:

- Medical records
- Legal documents
- Audit trails

Use Eventual Consistency when:**1. User-generated content:**

```
// Social media post - delays acceptable
@Transactional
public void createPost(Post post) {
    postRepository.save(post);
    eventPublisher.publish(new PostCreated(post));
    // Followers' feeds update eventually
}
```

2. Analytics and reporting:

```
// View counts, metrics - exact numbers not critical
@Async
public void incrementViewCount(Long postId) {
    statisticsRepository.incrementViews(postId);
}
```

3. Recommendations, search indexes, caches:

- Slightly stale data is acceptable
- Performance more important than accuracy

Decision criteria:**Business Impact Analysis:**

- What happens if data is temporarily inconsistent?
- Can users tolerate delays?
- What's the cost of being wrong?

Examples:**E-commerce checkout:**

- Payment processing: Strong consistency (financial)
- Email confirmation: Eventual consistency (can retry)
- Inventory: Strong consistency (prevent overselling)
- User reviews: Eventual consistency (delays OK)

Banking application:

- Account balance: Strong consistency
- Transaction history: Strong consistency
- Marketing recommendations: Eventual consistency
- Analytics dashboard: Eventual consistency

Social media:

- User authentication: Strong consistency
- Posts/comments: Eventual consistency
- Like counts: Eventual consistency
- Private messages: Medium consistency

Thank You! 🙏

Thank You for Reading! 🙏

Congratulations on completing this comprehensive guide on **Transaction Management in Spring Boot!**

We hope this handbook has provided you with valuable insights and practical knowledge that will help you excel in your interviews and advance your career as a Java/Spring Boot developer.

Transaction management is a critical aspect of enterprise application development, and mastering these concepts will set you apart as a skilled professional. Whether you're preparing for your first junior developer role or aiming for a technical lead position, we believe this material has equipped you with the confidence and expertise you need.

Stay Connected 📺

This is just the beginning! We regularly publish in-depth technical content, interview preparation guides, best practices, and hands-on tutorials to help developers like you grow and succeed.

For more such interesting knowledge and study materials, please visit:

🌐 <https://thegeekplanets.medium.com/>

Follow us to stay updated with:

- Interview preparation guides
- System design tutorials
- Spring Boot deep dives
- Microservices patterns
- Cloud technologies
- Best practices and real-world scenarios

We'd Love to Hear from You!

Your feedback helps us create better content. If you found this handbook helpful, please:

- Share it with fellow developers
- Leave your comments and suggestions
- Connect with us on Medium
- Suggest topics you'd like us to cover next

Best Wishes!

We wish you all the best in your interviews and career journey. Remember, continuous learning and practice are the keys to success in software development.

Keep learning, keep growing, and keep coding!

Happy Learning!

The Geek Planets Team

 <https://thegeekplanets.medium.com/>

"The expert in anything was once a beginner." - Helen Hayes